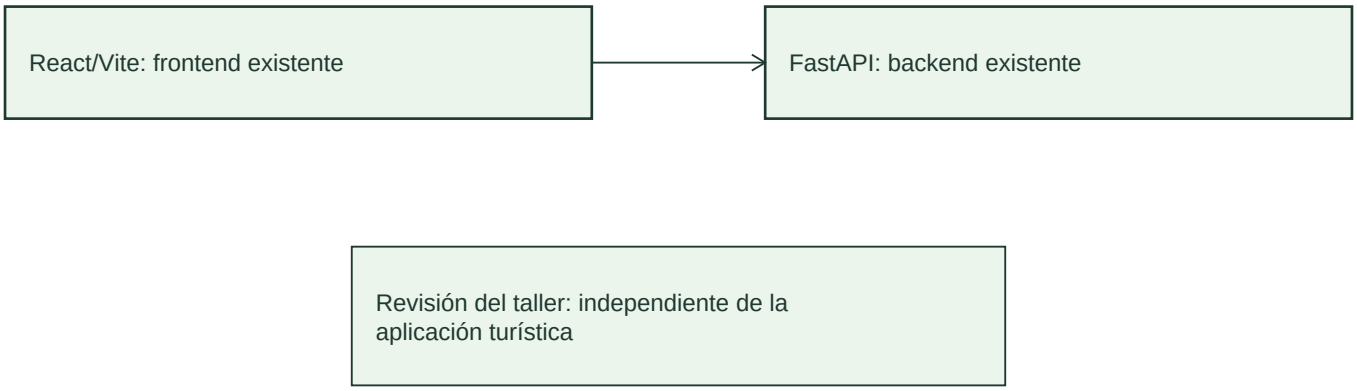
# OPITA 360 | Taller 5: Sistema de Agentes

## 1. Proyecto y herramienta

OPITA 360 es una aplicación turística del Huila. El frontend usa React, Vite y lucide-react; el backend utiliza Python, FastAPI y Pydantic. Los datos actuales están en memoria. PostgreSQL solo tiene un esquema preparado, sin conexión persistente.

El taller incorpora herramientas de desarrollo en Python, Ollama local y un servidor MCP externo. No cambia funciones turísticas, API, CORS ni esquema SQL. No se hacen commits ni push.

La especificación delimita revisión de código con evidencias auténticas. La planificación emite cuatro tareas con permisos fijos y contratos que vinculan tarea, agente y archivos. La procedencia del plan es Codex, sin atribuir a Copilot un documento inexistente.



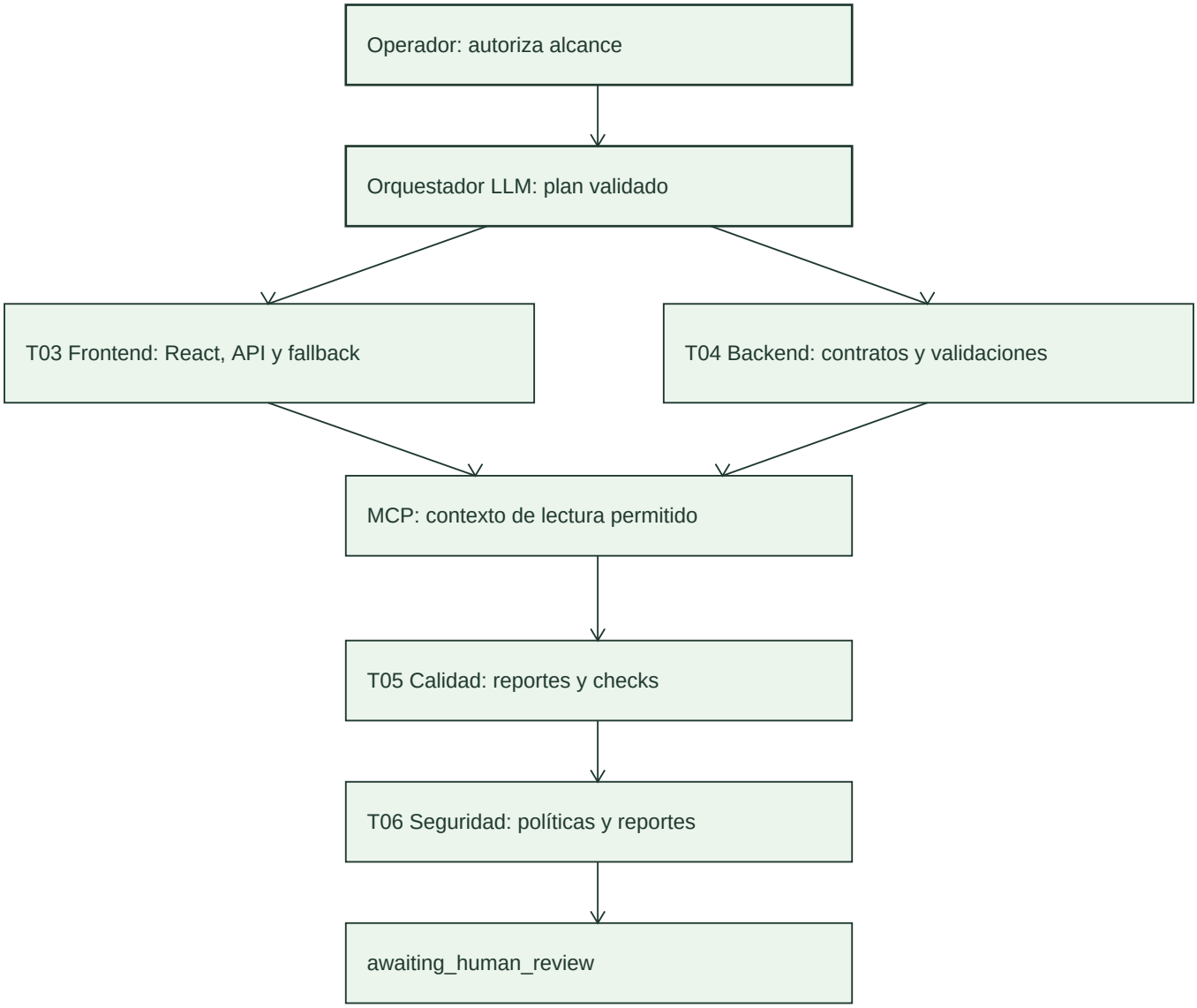

Herramientas: Python 3.10+, Ollama en 127.0.0.1:11434, modelo qwen3.5:4b y MCP por stdio. Node portátil 24.13.0 ejecuta el servidor externo sin sustituir Node del sistema. Los modelos no reciben shell ni edición turística.

Rúbrica del usuario: calidad del código 25%; agentes 20%; diagramas 20%; herramientas, permisos y modelo 15%; diferencias con referencia 10%; documento 10%. Este PDF contiene cuatro páginas dentro del máximo de ocho.

# OPITA 360 | Taller 5: Sistema de Agentes

## 2. Sistema de agentes

Los cinco roles usan qwen3.5:4b con contextos separados. Asignar un modelo no demuestra inferencia: solo las trazas reales permiten afirmar ejecución. El modo determinista no simula un LLM.



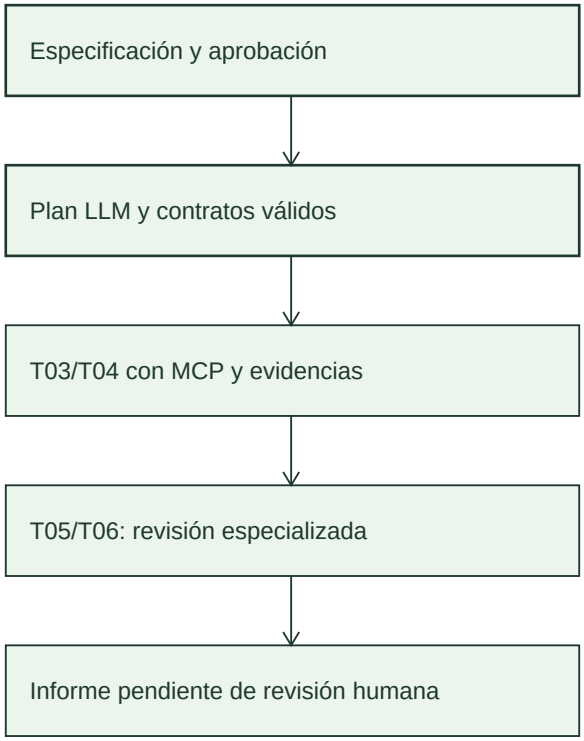

Frontend y backend reciben código leído realmente por MCP. Calidad revisa sus reportes y checks del operador; seguridad recibe políticas y resultados previos. Los hallazgos no se aplican automáticamente.

MCP: @modelcontextprotocol/server-filesystem@2.3.0 por stdio. Se exporta un archivo permitido, compara la lectura y verifica rechazo fuera del scope. Escrituras anunciadas no se autorizan. Staging no es sandbox OS.

# OPITA 360 | Taller 5: Sistema de Agentes

## 3. Calidad del código

La revisión conserva contratos y funcionalidades existentes. Las pruebas verifican tarea-agente, evidencia no vacía, duplicados, hashes, permisos y límites. El modelo puede errar: los hallazgos requieren contraste humano.




Resultados actuales: 27 pruebas de agentes y 8 de API aprobadas (35 total), compilaciones correctas. Chrome real: 11/14 antes y 14/14 después. Se corrigieron etiqueta de búsqueda, nombre del selector de días y estado ARIA del menú. Filtros, API, demo y teclado comprobados; registros y PNG en browser/.

Pruebas limitadas: nombres accesibles vía Chrome, etiquetas, teclado y ancho móvil; no certifican WCAG ni lector de pantalla. Las cinco inferencias y dos MCP son anteriores a la corrección. Se conserva checks-llm-historical.json; checks.json es actual. No se repitió LLM/MCP ni npm ci en esta actualización.

El LLM erró sobre validación de presupuesto y ausencia de pruebas MCP. T05/T06 no recibieron ledger MCP explícito; el asistente contrastó las afirmaciones con el host. Ver revision-hallazgos.md. Aceptación humana pendiente: la ejecución no prueba precisión semántica ni autoriza commit o push.

# OPITA 360 | Taller 5: Sistema de Agentes

## 4. Diferencias con el sistema de referencia

PENDIENTE: el sistema de referencia no está disponible. El usuario autorizó continuar dejando la comparación pendiente. No se inventan semejanzas o diferencias ni se afirma cumplimiento del 10% correspondiente.

Al recibir la fuente se contrastarán arquitectura, roles, modelos, permisos, MCP, aprobación humana y calidad. Cada diferencia debe citar archivos, decisiones o ejecuciones verificables de ambos sistemas.

Contraste interno distinto de la referencia: la versión previa inventariaba archivos con perfiles de modelo nulos. La corrección incorpora asignación local explícita, análisis LLM especializado, contexto MCP utilizado por frontend/backend, revisión de reportes previos y trazabilidad.

Ese contraste no atribuye propiedades a una referencia desconocida. Tampoco prueba ejecución por sí solo: los resultados auténticos se inspeccionan en los artefactos de la ejecución final.

Archivos para revisión: rubrica.md, especificacion.md, tareas.md, arquitectura.md, entrega.md, resultados.md y artefactos de pruebas/LLM. El ejecutor conserva la aplicación turística y PostgreSQL continúa sin conexión. Los pendientes deben permanecer visibles.

Antes de aceptar la entrega, una persona revisa el diff, las respuestas LLM, las comprobaciones reales y su correspondencia con el código. Una prueba no ejecutada o una aprobación automática no sustituye esa revisión.